# Supervyse OPF Hardware Test Utility User Guide

***Revision 1.11***

***12/2024***

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

# Contents


1 I2c-tools User Guide ..... 7
1.1 Introduction ..... **Error! Bookmark not defined.**
1.2 Command Usage ..... **Error! Bookmark not defined.**
1.2.1 Detect I2C chips ..... **Error! Bookmark not defined.**
1.2.2 Examine I2C registers ..... **Error! Bookmark not defined.**
1.2.3 Read from I2C/SMBus chip registers ..... **Error! Bookmark not defined.**
1.2.4 Set I2C registers ..... **Error! Bookmark not defined.**
1.2.5 Transfer I2C messages ..... **Error! Bookmark not defined.**
1.3 Command Example ..... **Error! Bookmark not defined.**

2 i3c-tools User Guide ..... 18
2.1 Introduction ..... 18
2.2 Command Usage ..... 18
2.2.1 Transfer I3C messages ..... 18
2.3 Command Example ..... 18

3 ncsi-tool User Guide ..... 20
3.1 Introduction ..... 20
3.2 Command Usage ..... 20
ncsi-tool 20
3.3 Command Format ..... 22
Generic Command ..... 22
Raw Command ..... 23
3.4 Generic Command Example ..... 24
Clear Initial State (0x00) ..... 24
Select Package (0x01) ..... 24
Deselect Package (0x02) ..... 24
Enable Channel (0x03) ..... 25
Disable Channel (0x04) ..... 25
Reset Channel (0x05) ..... 25
Enable Channel TX (0x06) ..... 25
Disable Channel TX (0x07) ..... 26
Set Link (0x09) ..... 27
Get Link Status (0x0A) ..... 27
Set VLAN Filter (0x0B) ..... 28
Enable VLAN (0x0C) ..... 28
Disable VLAN (0x0D) ..... 28
Enable Broadcast Filter (0x10) ..... 28
Disable Broadcast Filter (0x11) ..... 29
Get Version ID (0x15) ..... 29
Get Capabilities (0x16) ..... 30
Get NC-SI Pass-through Statistics (0x1A) ..... 30
3.5 Raw Command Example ..... 31
Example 1: Send control packet type 0x50 with appropriate raw command to Get Host's MAC address (System MAC Address). ..... 31
Example 2: Send NC-SI raw command to get channel capabilities.. 33

4 ncsi_test User Guide ..... 34
4.1 Introduction ..... 34
4.2 Command Usage ..... 34
4.3 Example ..... 35

5 AST2600 H/W Crypto User Guide ..... 38

5.1 Introduction ..... 38
5.2 User Space Test Utilities ..... 38

6 lpc_cmds User Guide ..... 40
6.1 Introduction ..... 40
6.2 KCS Driver ..... 40
6.3 Snoop Driver ..... 42
Example I 42
6.4 SIO Driver ..... 43
Example I 44
6.5 Mailbox Driver ..... 45
Command Usage ..... 45
Example I 45

7 Register Access Tool User Guide ..... 46
7.1 AST2600 I/O Map ..... 46
7.2 io shell ..... 47

8 SPI Tool User Guide ..... 49
8.1 mtd_debug ..... 49
8.2 mtdinfo ..... 49

9 pwm_tool User Guide ..... 51
9.1 Introduction ..... 51
9.2 Command Usage ..... 52

10 mdio-tool User Guide ..... 54
10.1 Introduction ..... 54
10.2 Command Usage ..... 54
10.3 Example ..... 54

11 iperf3 User Guide ..... 55
11.1 Introduction ..... 55
11.2 Command Usage ..... **Error! Bookmark not defined.**
11.3 Example ..... 56

12 eth-util User Guide ..... 57
12.1 Introduction ..... 57
12.2 Command Usage ..... 57
12.3 Example ..... 57

13 hwclock User Guide ..... 58
13.1 Introduction ..... 58
13.2 Command Usage ..... 58
13.3 Usage examples and precautions ..... 58
13.4 Precautions ..... 59

14 adc_test User Guide ..... 61
14.1 Introduction ..... 61
14.2 Command Usage ..... 61
14.3 Example ..... 61

# Revision History

| Revision Number | Date | Author | Reviewer | Summary |
|---|---|---|---|---|
| 1.0 | 2023-06-28 | - | - | RV23.05 Release Document |
| 1.1 | 2023-09-26 | Nick Lin | Edwin Liu | The user guide of AST2600 hardware crypto tool |
| 1.2 | 2023-09-27 | Austin Su | Roy Wang | Add Supervyse OPF lpc_cmds User Guide |
| 1.3 | 2024-02-17 | Jimmy He | Allen Liu | Add ncsi_test User Guide |
| 1.4 | 2024-04-16 | Nick Lin | Derick Lai | Register Access Tool and SPI Test Utilities are added. |
| 1.5 | 2024-07-01 | JoeYJ Yang | Edwin Liu | Add Supervyse OPF i3c-tools User Guide and modify I2c-tools User Guide |
| 1.6 | 2024-07-02 | Alice Chen | Edwin Liu | Add Supervyse OPF pwm_tool User Guide |
| 1.7 | 2024-07-17 | Jimmy He | Edwin Liu | Add mdio-tool User Guide |
| 1.8 | 2024-07-17 | Jimmy He | Edwin Liu | Add iperf3 User Guide |
| 1.9 | 2024-07-19 | Jimmy He | Edwin Liu | Add eth-util User Guide |
| 1.10 | 2024-07-26 | Sam Tsai | Nick Lin | Add hwclock User Guide |
| 1.11 | 2024-12-17 | Jimmy He | Edwin Liu | Update the content of ncsi-tool/ncsi_test User Guide |

# 1 i2c-tools User Guide

## 1.1 Introduction

This package (i2c-tools) contains a heterogeneous set of I2C tools for Linux: a bus probing tool, a chip dumper, register-level access helpers, EEPROM decoding scripts, and more.

In this section, all I2C device addresses are represented in a **7-bit format**.

However, schematics commonly use the 8-bit format (7-bit address shifted left by 1).

Be careful to avoid confusion between the two formats.

For more detail, please refer to [**Package i2c-tools**].

## 1.2 i2cdetect

i2cdetect is a userspace program to scan an I2C bus for devices.

### 1.2.1 Command Usage

```
Usage: i2cdetect [-y] [-a] [-q|-r] I2CBUS [FIRST LAST]
       i2cdetect -F I2CBUS
       i2cdetect -l
  I2CBUS is an integer or an I2C bus name
  If provided, FIRST and LAST limit the probing range.
```

**-y** Disable interactive mode. By default, i2cdetect will wait for a confirmation from the user before messing with the I2C bus. When this flag is used, it will perform the operation directly. This is mainly meant to be used in scripts.

**-a** Force scanning of non-regular addresses. Not recommended.

**-q** Use SMBus "quick write" commands for probing (by default, the command used is the one believed to be the safest for each address). Not recommended. This is known to corrupt the Atmel AT24RF08 EEPROM found on many IBM Thinkpad laptops.

**-r** Use SMBus "read byte" commands for probing (by default, the command used is the one believed to be the safest for each address). Not recommended. This is known to lock SMBus on various write-only chips (most notably clock chips at address 0x69).

**I2CBUS:** Indicates the number of the I2C bus to be used.

**FIRST, LAST:** Restrict the scanning range (default: from 0x03 to 0x77).

**-F** Display the list of functionalities implemented by the adapter and exit.

**-l** Output a list of installed busses.

**Interpreting the Output**

Each cell in the output table will contain one of the following symbols:

* "--". The address was probed but no chip answered.

* "UU". Probing was skipped, because this address is currently in use by a driver. This strongly suggests that there is a chip at this address.

* An address number in hexadecimal, e.g. "2d" or "4e". A chip was found at this address.

**Default probing method**

For addresses in the range **0x30 to 0x37** and **0x50 to 0x5F**, the "**read byte**" command is used for probing.

For all **other addresse**s, the "**quick write**" command is used.

## 1.2.2 Usage Example

**List all installed I2C buses using the following command:**

```
# i2cdetect -l
i2c-0   i2c             1e78a080.i2c-bus                        I2C adapter
i2c-1   i2c             1e78a100.i2c-bus                        I2C adapter
i2c-2   i2c             1e78a180.i2c-bus                        I2C adapter
i2c-3   i2c             1e78a200.i2c-bus                        I2C adapter
i2c-4   i2c             1e78a280.i2c-bus                        I2C adapter
i2c-5   i2c             1e78a300.i2c-bus                        I2C adapter
i2c-6   i2c             1e78a380.i2c-bus                        I2C adapter
i2c-7   i2c             1e78a400.i2c-bus                        I2C adapter
i2c-8   i2c             1e78a480.i2c-bus                        I2C adapter
i2c-9   i2c             1e78a500.i2c-bus                        I2C adapter
i2c-11  i2c             1e78a600.i2c-bus                        I2C adapter
i2c-12  i2c             1e78a680.i2c-bus                        I2C adapter
i2c-13  i2c             1e78a700.i2c-bus                        I2C adapter
i2c-14  i2c             1e78a780.i2c-bus                        I2C adapter
i2c-15  i2c             1e78a800.i2c-bus                        I2C adapter
i2c-24  i2c             i2c-13-mux (chan_id 0)                  I2C adapter
i2c-25  i2c             i2c-5-mux (chan_id 0)                   I2C adapter
i2c-26  i2c             i2c-5-mux (chan_id 1)                   I2C adapter
i2c-27  i2c             i2c-5-mux (chan_id 2)                   I2C adapter
i2c-28  i2c             i2c-5-mux (chan_id 3)                   I2C adapter
i2c-29  i2c             i2c-6-mux (chan_id 0)                   I2C adapter
i2c-30  i2c             i2c-6-mux (chan_id 1)                   I2C adapter
i2c-31  i2c             i2c-6-mux (chan_id 2)                   I2C adapter
i2c-32  i2c             i2c-6-mux (chan_id 3)                   I2C adapter
i2c-33  i2c             i2c-7-mux (chan_id 0)                   I2C adapter
i2c-34  i2c             i2c-7-mux (chan_id 1)                   I2C adapter
i2c-35  i2c             i2c-7-mux (chan_id 2)                   I2C adapter
i2c-36  i2c             i2c-7-mux (chan_id 3)                   I2C adapter
i2c-37  i2c             i2c-12-mux (chan_id 0)                  I2C adapter
i2c-38  i2c             i2c-12-mux (chan_id 1)                  I2C adapter
i2c-39  i2c             i2c-12-mux (chan_id 2)                  I2C adapter
i2c-40  i2c             i2c-12-mux (chan_id 3)                  I2C adapter
```

**Display the list of functionalities implemented on i2c-2 using the following command:**

```
# i2cdetect -F 2
Functionalities implemented by /dev/i2c-2:
I2C                              yes
SMBus Quick Command              yes
SMBus Send Byte                  yes
SMBus Receive Byte               yes
SMBus Write Byte                 yes
SMBus Read Byte                  yes
SMBus Write Word                 yes
SMBus Read Word                  yes
SMBus Process Call               yes
SMBus Block Write                yes
SMBus Block Read                 yes
SMBus Block Process Call         no
SMBus PEC                        yes
I2C Block Write                  yes
I2C Block Read                   yes
```

**Detect all slave devices on i2c-2 in non-interactive mode using the following command:**

```
# i2cdetect -y 2
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- 0a -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- 4e --
50: UU -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: 60 -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

**Detect all slave devices on i2c-2 in non-interactive mode, restricting the range to addresses between 0x50 and 0x6F, using the following command:**

```
# i2cdetect -y 2 0x50 0x6F
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:
10:
20:
30:
40:
50: UU -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: 60 -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:
70:
```

## 1.3 i2cdump

i2cdump is a small helper program to examine registers visible through the I2C bus.

### 1.3.1 Command Usage

```
Usage: i2cdump [-f] [-y] [-r first-last] [-a] I2CBUS ADDRESS [MODE]
  I2CBUS is an integer or an I2C bus name
  ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
  MODE is one of:
    b (byte, default)
    w (word)
    W (word on even register addresses)
    s (SMBus block, deprecated)
    i (I2C block)
    c (consecutive byte)
    d (double word)
    Append p for SMBus PEC
```

**-f** Force access to the device even if it is already busy. By default, i2cdump will refuse to access a device which is already under the control of a kernel driver. Using this flag is dangerous, it can seriously confuse the kernel driver in question. It can also cause i2cdump to return invalid results. So use at your own risk and only if you know what you're doing.

**-y** Disable interactive mode. By default, i2cdump will wait for a confirmation from the user before messing with the I2C bus. When this flag is used, it will perform the operation directly. This is mainly meant to be used in scripts.

**-r first-last** Limit the range of registers being accessed. This option is not available with mode **s**. For mode **W**, **first** must be even and **last** must be odd.

**-a** Allow using addresses between 0x00 - 0x07 and 0x78 - 0x7f. Not recommended.

**I2CBUS:** Indicates the number of the I2C bus to be used.

**ADDRESS:** I2C device address.

**MODE:**

**b** Reads data in "single byte" mode. If the mode is not specified, this is the default mode.

**w** Reads data in "16-bit word" mode (two bytes).

**s** Reads data in "SMBus block" mode; the reading follows the SMBus block specification.

**i** Reads data in "I2C block" mode; used for performing I2C block reads.

**c** Reads data in "consecutive bytes" mode. This is commonly used for chips with an auto-increment address feature (e.g., EEPROMs) that automatically increment the address and read multiple data bytes in one operation.

**W** Similar to w, but read commands are issued only on even register addresses. This mode is mainly used for specific devices such as EEPROMs.

A **p** can also be appended to the mode parameter (except for i and W) to enable PEC (Packet Error Check). If the mode parameter is omitted, i2cdump defaults to byte access without PEC.

**Warning**

i2cdump can be dangerous if used improperly. Most notably, the c mode starts with WRITING a byte to the chip. On most chips it will be stored in the address pointer register, which is OK, but some chips with a single register or no (visible) register at all will most likely see this as a real WRITE, resulting in possible misbehavior or corruption. Do not use i2cdump on random addresses. Anyway, it is of little use unless you have good knowledge of the chip you're working with and an idea of what you are looking for.

### 1.3.2 Usage Example

**Dump the entire contents of the I2C device at address 0x50 on i2c-1, using the default byte-read mode, and skip user confirmation with the following command:**

```
# i2cdump -y 1 0x50
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: ab 65 6c 6c 6f 20 45 45 50 52 4f 4d 0a 83 18 87    ?ello EEPROM????
10: ab 8b aa 83 18 87 88 8c ff ff ff ff ff ff ff ff    ????????........
20: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
30: 12 34 56 78 ff ff ff ff ff ff ff ff ff ff ff ff    ?4Vx............
40: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
50: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
60: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
70: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
80: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
90: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
a0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
b0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
c0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
d0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
e0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
f0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
```

**Dump registers 0x30 to 0x34 of the I2C device at address 0x50 on i2c-1, using the default byte-read mode, and skip user confirmation with the following command:**

```
# i2cdump -y -r 0x30-0x34 1 0x50
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
30: 12 34 56 78 ff                                     ?4Vx.
```

## 1.4 i2cget

i2cget is a small helper program to read registers visible through the I2C bus (or SMBus).

### 1.4.1 Command Usage

```
Usage: i2cget [-f] [-y] [-a] I2CBUS CHIP-ADDRESS [DATA-ADDRESS [MODE [LENGTH]]]
  I2CBUS is an integer or an I2C bus name
  ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
  MODE is one of:
    b (read byte data, default)
    w (read word data)
    c (write byte/read byte)
    s (read SMBus block data)
    i (read I2C block data)
    Append p for SMBus PEC
  LENGTH is the I2C block data length (between 1 and 32, default 32)
```

**-f** Force access to the device even if it is already busy. By default, i2cget will refuse to access a device which is already under the control of a kernel driver. Using this flag is dangerous, it can seriously confuse the kernel driver in question. It can also cause i2cget to return an invalid value. So use at your own risk and only if you know what you're doing.

**-y** Disable interactive mode. By default, i2cget will wait for a confirmation from the user before messing with the I2C bus. When this flag is used, it will perform the operation directly. This is mainly meant to be used in scripts. Use with caution.

**-a** Allow using addresses between 0x00 - 0x07 and 0x78 - 0x7f. Not recommended.

**I2CBUS:** Indicates the number of the I2C bus to be used.

**CHIP-ADDRESS:** I2C device address.

**DATA-ADDRESS**: The address on the device from which to read data.

**MODE:**

**b** Reads data in "single byte" mode (default).

**w** Reads data in "16-bit word" mode.

**c** Performs a "write byte/read byte" operation. This mode writes a byte to the DATA-ADDRESS and immediately reads a single byte from it.

**s** Reads data in "SMBus block" mode.

**i** Reads data in "I2C block" mode.

A **p** can also be appended to the mode parameter (except for i and W) to enable PEC (Packet Error Check). If the mode parameter is omitted, i2cdump defaults to byte access without PEC.

**LENGTH:** The length parameter, if applicable and specified, sets the length of the block transaction. Valid values are between 1 and 32. Default value is 32.

**Warning**

i2cget can be extremely dangerous if used improperly. I2C and SMBus are designed in such a way that an SMBus read transaction can be seen as a write transaction by certain chips. This is particularly true if setting mode to cp (write byte/read byte with PEC). Be extremely careful using this program.

### 1.4.2 Usage Example

**Read the first 8 bytes of an EEPROM device at the 7-bit address 0x50 on i2c-1 using the following command:**

```
# i2cget -y 1 0x50 0x00 i 8
0xab 0x65 0x6c 0x6c 0x6f 0x20 0x45 0x45
```

## 1.5 i2cset

i2cset is a small helper program to set registers visible through the I2C bus.

### 1.5.1 Command Usage

```
Usage: i2cset [-f] [-y] [-m MASK] [-r] [-a] I2CBUS CHIP-ADDRESS DATA-ADDRESS [VALUE]
... [MODE]
  I2CBUS is an integer or an I2C bus name
  ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
  MODE is one of:
    c (byte, no value)
    b (byte data, default)
    w (word data)
    i (I2C block data)
    s (SMBus block data)
    Append p for SMBus PEC
```

**-f** Force access to the device even if it is already busy. By default, i2cset will refuse to access a device which is already under the control of a kernel driver. Using this flag is dangerous, it can seriously confuse the kernel driver in question. It can also cause i2cset to silently write to the wrong register. So use at your own risk and only if you know what you're doing.

**-y** Disable interactive mode. By default, i2cset will wait for a confirmation from the user before messing with the I2C bus. When this flag is used, it will perform the operation directly. This is mainly meant to be used in scripts.

**-m mask** The mask parameter, if specified, describes which bits of value will be actually written to data-address. Bits set to 1 in the mask are taken from value, while bits set to 0 will be read from data-address and thus preserved by the operation. Please note that this parameter assumes that the read and write operations for the specified mode are symmetrical for the device you are accessing. This may or may not be the case, as neither I2C nor SMBus guarantees this.

**-r** Read back the value right after writing it, and compare the result with the value written. This used to be the default behavior. The same limitations apply as those of option -m.

**-a** Allow using addresses between 0x00 - 0x07 and 0x78 - 0x7f. Not recommended.

**I2CBUS:** Indicates the number of the I2C bus to be used.

**CHIP-ADDRESS:** I2C device address.

**DATA-ADDRESS**: Specifies the address on the chip to write to.

**VALUE:** If specified, is the value to write to that location on the chip. If this parameter is omitted, then a **short write** (writing only the DATA-ADDRESS to the chip) is issued. For most chips, it simply sets an internal pointer to the target location, but doesn't actually write to that location. For a few chips though, in particular simple ones with a single register, this short write is an actual write. If the mode parameter is s or i, multiple values can be specified.

**MODE:**

**c** Sends a single byte to the device without including a data value.

**b** Writes a single byte of data to the specified address, this is the default mode.

**w** Writes a 16-bit word (2 bytes) of data to the specified address.

**i** Writes a block of data using the I2C block protocol.

**s** Writes a block of data using the SMBus block protocol.

A **p** can also be appended to the mode parameter (except for i and W) to enable PEC (Packet Error Check). If the mode parameter is omitted, i2cdump defaults to byte access without PEC.

**Warning**

i2cset can be extremely dangerous if used improperly. It can confuse your I2C bus, cause data loss, or have more serious side effects. Writing to a serial EEPROM (SPD, Serial Presence Detect) on a memory DIMM (chip addresses between 0x50 and 0x57) may DESTROY your memory, leaving your system unbootable! Be extremely careful using this program.

### 1.5.2 Usage Example

**Write the value 0x87 to the 8-bit register 0x40 of the I2C device at the 7-bit address 0x50 on i2c-1, skipping user confirmation, using the following command:**

```
# i2cset -y 1 0x50 0x40 0x87
```

## 1.6 i2ctransfer

i2ctransfer is a program to create I2C messages and send them combined as one transfer.

### 1.6.1 Command Usage

```
Usage: i2ctransfer [-f] [-y] [-v] [-V] [-a] I2CBUS DESC [DATA] [DESC [DATA]]...
  I2CBUS is an integer or an I2C bus name
  DESC describes the transfer in the form: {r|w}LENGTH[@address]
    1) read/write-flag 2) LENGTH (range 0-65535, or '?')
    3) I2C address (use last one if omitted)
  DATA are LENGTH bytes for a write message. They can be shortened by a suffix:
    = (keep value constant until LENGTH)
    + (increase value by 1 until LENGTH)
    - (decrease value by 1 until LENGTH)
    p (use pseudo random generator until LENGTH with value as seed)
```

**-f** Force access to the device even if it is already busy. By default, i2ctransfer will refuse to access a device which is already under the control of a kernel driver. Using this flag is dangerous, it can seriously confuse the kernel driver in question. It can also cause i2ctransfer to silently write to the wrong register. So use at your own risk and only if you know what you're doing.

**-y** Disable interactive mode. By default, i2ctransfer will wait for a confirmation from the user before messing with the I2C bus. When this flag is used, it will perform the operation directly. This is mainly meant to be used in scripts.

**-v** Enable verbose output. It will print infos about all messages sent, i.e. not only for read messages but also for write messages.

**-V** Display the version and exit.

**-a** Allow using addresses between 0x00 - 0x07 and 0x78 - 0x7f. Not recommended.

**I2CBUS:** Indicates the number of the I2C bus to be used.

**DESC:** Consists of multiple descriptor blocks, where each block is structured as follows:

**{r|w}<length_of_message>[@address]**

**{r|w}:** specifies if the message is read or write.

**<length_of_message>:** specifies the number of bytes read or written in this message.

**[@address]:** specifies the 7-bit address of the chip to be accessed for this message, and is an integer.

**DATA:** If the I2C message is a write, then a data block with the data to be written follows.

It consists of <length_of_message> bytes which can be marked with the usual prefixes for hexadecimal, octal, etc. To make it easier to create larger data blocks easily, the data byte can have a suffix.

**=** keep value constant until end of message (i.e. 0= means 0, 0, 0, ...)

**+** increase value by 1 until end of message (i.e. 0+ means 0, 1, 2, ...)

**-** decrease value by 1 until end of message (i.e. 0xff- means 0xff, 0xfe, 0xfd, ...)

**p** use value as seed for an 8 bit pseudo random sequence (i.e. 0p means 0x00, 0x50, 0xb0, ...)

**Warning**

i2ctransfer can be extremely dangerous if used improperly. It can confuse your I2C bus, cause data loss, or have more serious side effects. Writing to a serial EEPROM (SPD, Serial Presence Detect) on a memory DIMM (chip addresses between 0x50 and 0x57) may DESTROY your memory, leaving your system unbootable! Be extremely careful using this program.

### 1.6.2 Usage Example

**On bus 1, from an EEPROM at address 0x50, read 8 byte from offset 0x10 (first message writes one byte to set the memory pointer to 0x10, second message reads from the same chip):**

```
# i2ctransfer -y 1 w1@0x50 0x10 r8
0xab 0x8b 0xaa 0x83 0x18 0x87 0x88 0x8c
```

**For the same EEPROM, at offset 0x50 write 0xff 0xfe … 0xf0 (one write message; first byte sets the memory pointer to 0x50, 0xff is the first data byte, all following data bytes are decreased by one):**

```
# i2ctransfer 1 w17@0x50 0x50 0xff-
```

# 2 i3c-tools User Guide

## 2.1 Introduction

This package has the i3ctransfer tool used to send private SDR transfer over I3C bus to a particular device exposed via i3cdev module to /dev folder. The i3ctransfer tool allows to construct and concat multiple I3C private transfer into a single transfers. It aims to help developer during the I3C HC and I3C device drivers development.

## 2.2 Command Usage

### 2.2.1 Transfer I3C messages

i3ctransfer [-d] /dev/I3CDEV [-w] [DATA] [-r] [LENGTH]

- I3CDEV I3C device name
- -d Device to use.
- -r Read data length.
- -w Write data block.

## 2.3 Command Example

```
## Write one data byte in single private transfer
# i3ctransfer -d /dev/I3CDEV -w 0xde

## Write multiple data bytes in single private transfer
# i3ctransfer -d /dev/I3CDEV -w "0xde,0xad,0xbe,0xef"

## Write multiple data bytes in multiple private transfers
# i3ctransfer -d /dev/I3CDEV -w "0xde,0xad" -w "0xbe,0xef"

## Read multiple data bytes in single private transfer
# i3ctransfer -d /dev/I3CDEV -r <data length>

## Read multiple data bytes in multiple private transfer
# i3ctransfer -d /dev/I3CDEV -r <data length> -r <data length>

## Read and write multiple data bytes in multiple private transfer
# i3ctransfer -d /dev/I3CDEV -w "0xde,0xad,0xbe,0xef" -r <data length>

## Parameters:
## i3c dev: I3C device exposed by i3cdev under /dev folder
```

## data length: Data lenth to read on this message

# 3 ncsi-tool User Guide

## 3.1 Introduction

∗ ncsi-tool is a tool which can let users send NC-SI commands for debugging.

∗ ncsi-tool uses the Linux NetLink API to communicate with the Linux NCSI stack.

∗ When you need to test the Linux Kernel NC-SI stack, particularly the BMC's low-level NC-SI functionality, use this tool.

∗ ncsi-tool is used for NC-SI over RBT, **NOT for NC-SI over MCTP**.

For more details about the NC-SI command format, refer to [DSP0222 NC-SI Specification].

## 3.2 Command Usage

### ncsi-tool

The ncsi-tool can be found in the /usr/bin filesystem to let us send NC-SI generic commands or NC-SI raw commands.

```
# ncsi-tool
Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
Usage: ncsi-tool [option(s)]
--clear_initial_state         : Clear Initial State (0x00)
--select_package              : Select   Package    (0x01), default H/W arbitration as
[Disable]
--deselect_package            : Deselect Package    (0x02)
--enable_channel              : Enable   Channel    (0x03)
--disable_channel             : Disable  Channel    (0x04)
--reset_channel               : Reset    Channel    (0x05)
--enable_channel_tx           : Enable   Channel TX (0x06)
--disable_channel_tx          : Disable  Channel TX (0x07)
--set_link "[setting]"        : Set link            (0x09)
--get_link_status             : Get Link Status     (0x0A)
--set_vlan_filter             : Set VLAN Filter     (0x0B)
                                Default as "disable VLAN" filter if --
enable_vlan_filter or --disable_vlan_filter is not specified in the command line

--enable_vlan [vlan mode]     : Enable  VLAN        (0x0C)
                                vlan mode : 0x1(VLAN only), 0x2(VLAN + non-VLAN),
0x3(Any VLAN + non-VLAN)

--disable_vlan                : Disable VLAN       (0x0D)
--enable_bcast_filter [filter]: Enable  Broadcast Filter (0x10)
--disable_bcast_filter        : Disable Broadcast Filter (0x11)
--get_version_id              : Get Version ID           (0x15)
--get_capabilities            : Get Capabilities         (0x16)
--get_pass_thru_stats         : Get NC-SI Pass-through Statistics (0x1A)
--raw "[data0] [data1] ..."   : Send NC-SI raw command, max length is 16
```

```
--nic_idx                      : (optional) Assign NIC index, e.g. '0' for ncsi0, '1'
for ncsi1, default as value 0(aka ncsi0) if not specified in the command line
--package                      : (optional) Assign package ID, value started from 0,
default as 0 if not specified in the command line
--channel                      : (optional) Assign channel ID, value started from 0,
default as 0 if not specified in the command line
--enable_vlan_filter           : (optional) Enable  VLAN Filter
--disable_vlan_filter          : (optional) Disable VLAN Filter
--ctrl_packet_type [type]      : Assign "Contorl Packet Type"
--help                         : Show this help message

e.g. Send NC-SI raw command with "Control Packet Type" - 0x16.
# ncsi-tool --raw "" --ctrl_packet_type 0x16

e.g. Send NC-SI command "Get Capabilities" - 0x16.
# ncsi-tool --get_capabilities
```

## 3.3 Command Format

The command format for ncsi-tool can be separated into two parts: **Generic Command** and **Raw Command**.

**Generic Command**: This means ncsi-tool will handle the NC-SI command data format for you.

**Raw Command:** This means ncsi-tool will not handle the NC-SI command data format. You must manually fill in the data by referring to the NC-SI Specification.

### Generic Command

```
# ncsi-tool [--generic_cmd] [--optional_info]
```

**--optional_info**

- The optional_info is listed below. The default value will be 0 if not specified in the commandline.
    - --package ${number}: To assign the Package ID of the NCSI device.
    - --channel ${number}: To assign the Channel ID of the NCSI device.
    - --nic_idx ${number}: To assign the index of NCSI controller. For AST2600 SoC, there are two NCSI controllers. ncsi0 & ncsi1.0 for ncsi0, 1 for ncsi1.

**If you only want to check whether NC-SI is working, we recommend using the following commands for testing:**

```
        --get_link_status              : Get Link Status      (0x0A)
        --get_version_id               : Get Version ID       (0x15)
        --get_capabilities             : Get Capabilities     (0x16)
```

**Table 3-1 Description of ncsi-tool Generic Command Support for NC-SI Commands**

| NC-SI Command Name | Description |
|---|---|
| Clear Initial State | Used by the Management Controller to acknowledge that the Network Controller is in the Initial State |
| Select Package | Used to explicitly select a controller package to transmit packets through the NC-SI interface |
| Deselect Package | Used to explicitly instruct the controller package to stop transmitting packets through the NC-SI interface |
| Enable Channel | Used to enable the NC-SI channel and to cause the forwarding of bidirectional Management Controller packets to start |
| Disable Channel | Used to disable the NC-SI channel and to cause the forwarding of bidirectional Management Controller packets to cease |
| Reset Channel | Used to synchronously put the Network Controller back to the Initial State |
| Enable Channel Network TX | Used to explicitly enable the channel to transmit Pass-through packets onto the network |
| Disable Channel Network TX | Used to explicitly disable the channel from transmitting Pass-through packets onto the network |
| Set Link | Used during OS absence to forcelink settings, or to return to auto-negotiation mode |
| **Get Link Status** | **Used to get current link status information** |
| Set VLAN Filter | Used to program VLAN IDs for VLAN filtering |
| Enable VLAN | Used to enable VLAN filtering of Management Controller RX packets |
| Disable VLAN | Used to disable VLAN filtering |
| Enable Broadcast Filter | Used to enable selective broadcast packet filtering |
| Disable Broadcast Filter | Used to disable all broadcast packet filtering, and to enable the forwarding of all broadcast packets |
| **Get Version ID** | **Used to get controller-related version information** |
| **Get Capabilities** | **Used to get optional functions supported by the NC-SI** |
| Get NC-SI Pass-through Statistics | Used to get current packet statistics for the Ethernet Controller |

## Raw Command

- The raw command format looks like the example below. --raw and --ctrl_packet_type are REQUIRED options.

```
# ncsi-tool --raw "[DATA0] [DATA1] [DATA2] ... [DATA15]" --ctrl_packet_type XX [--
optional_info]
```

**--raw "[DATA0] [DATA1] ... [DATA15]"**

- The NC-SI raw command, and the maximum length is 16. unit: byte

**--ctrl_packet_type XX**

- "Control Packet Type" is an essential filed in the NC-SI command protocol.

**--optional_info**

- The optional_info are listed below. The default value will be 0 if not specified in the commandline.
    - --package ${number}: To assign the Package ID of the NCSI device.
    - --channel ${number}: To assign the Channel ID of the NCSI device.
    - --nic_idx ${number}: To assign the index of NCSI controller. For AST2600 SOC, there are two NCSI controllers. ncsi0 & ncsi1.0 for ncsi0, 1 for ncsi1.

## 3.4 Generic Command Example

### Clear Initial State (0x00)

```
# ncsi-tool --clear_initial_state --nic_idx 0

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Clear Initial State
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

### Select Package (0x01)

```
# ncsi-tool --select_package --nic_idx 1

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-1 chan(00:00)
- send from ncsi1
Select Package
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

### Deselect Package (0x02)

```
# ncsi-tool --deselect_package

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.
```

```
Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Deselect Package
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Enable Channel (0x03)

```
# ncsi-tool --enable_channel

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Enable Channel
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Disable Channel (0x04)

```
# ncsi-tool --disable_channel

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Disable Channel
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Reset Channel (0x05)

```
# ncsi-tool --reset_channel

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Reset Channel
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Enable Channel TX (0x06)

```
# ncsi-tool --enable_channel_tx

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Enable Channel Network TX
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Disable Channel TX (0x07)

```
# ncsi-tool --disable_channel_tx

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Disable Channel Network TX
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Set Link (0x09)

```
# ncsi-tool --set_link 0x0006F26F

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Set Link
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

- For more info, please refer to NC-SI Spec: DSP0222_1.1.0.pdf
- Table 40 - Set Link command packet format.
- Table 41 - Set Link bit definition.

## Get Link Status (0x0A)

```
# ncsi-tool --get_link_status

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Get Link Status
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
NC-SI Link Status : 0x6F26F
        Link                    : up
        Speed and duplex        : 1000BASE-T full duplex
        Auto Negotiate Flag     : enable
        Auto Negotiate Complete : done
        Parallel Detection Flag : disable
        TX Flow Control         : disable
        RX Flow Control         : enable
        SerDes Link             : down
        OEM Link Speed Valid    : no
        Link Partner Adv. Flow Control : Symmetric Pause
        1000BASE-T full duplex         : support
        1000BASE-T half duplex         : N/A
        100BASE-T4                     : N/A
        100BASE-TX full duplex         : support
        100BASE-TX half duplex         : support
        10BASE-TX  full duplex         : support
        10BASE-TX  half duplex         : support
```

## Set VLAN Filter (0x0B)

```
# ncsi-tool --set_vlan_filter [VLAN index] [--enable_vlan_filter] or [--
disable_vlan_filter]
```

- VLAN index: VLAN index started from 1.
- --enable_vlan_filter : Enable this VLAN filter.
- --disable_vlan_filter: Disable this VLAN filter.
- Enable VLAN filter #0x1.

```
# ncsi-tool --set_vlan_filter 0x1 --enable_vlan_filter
Set VLAN Filter
```

- Disable VLAN filter #0x2.

```
# ncsi-tool --set_vlan_filter 0x2 --disable_vlan_filter
Set VLAN Filter
```

- For more info, please refer to NC-SI Spec: **DSP0222_1.1.0_NC-SI Spec.pdf**.
- Table 52 - Set VLAN Filter command packet format
- Table 53 - Possible Settings for Filter Selector field (8-bit field)
- Table 54 - Possible Settings for Enable(E) field (1-bit field)

## Enable VLAN (0x0C)

```
# ncsi-tool --enable_vlan [VLAN mode]
Enable VLAN
```

- VLAN mode "0x1": VLAN only
- VLAN mode "0x2": VLAN + non-VLAN
- VLAN mode "0x3": Any VLAN + non-VLAN
- Enable VLAN with VLAN mode 0x2(VLAN + non-VLAN)

```
# ncsi-tool --enable_vlan 0x2
Enable VLAN
```

- For more info, please refer to NC-SI Spec: **DSP0222_1.1.0_NC-SI Spec.pdf**.
- Table 57 - Enable VLAN command packet format
- Table 58 - VLAN Enable modes

## Disable VLAN (0x0D)

```
# ncsi-tool --disable_vlan
Disable VLAN
```

## Enable Broadcast Filter (0x10)

```
# ncsi-tool --enable_bcast_filter [Filter Settings]
Enable Broadcast Filter
```

- [Filter Settings]: 32-bit integer of filter settings.

- For more info, please refer to NC-SI Spec: **DSP0222_1.1.0_NC-SI Spec.pdf**.
- Table 68 - Enable Broadcast Filter command packet format
- Table 69 - Broadcast Packet Filter Settings filed

```
# ncsi-tool --enable_bcast_filter 0xF
Enable Broadcast Filter
```

## Disable Broadcast Filter (0x11)

```
# ncsi-tool --disable_bcast_filter

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Disable Broadcast Filter
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
```

## Get Version ID (0x15)

```
# ncsi-tool --get_version_id

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Get Version ID
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
NC-SI: Get Version ID :
        ver_major  = 0xF1
        ver_minor  = 0xF0
        fw_name    = [I210  FW ver]
        fw_version = 0x3000701
        pci_did    = 0x1533
        pci_vid    = 0x8086
        pci_ssid   = 0x35BC
        pci_svid   = 0x8086
```

## Get Capabilities (0x16)

```
# ncsi-tool -get_capabilities

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Get Capabilities
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
NC-SI: Channel capabilities :
        cap_flags        = 0x0000001F
        bcast_filter     = 0x0000000F
        mcast_filter     = 0x00000007
        buffering        = 0x00001FF0
        aen_support      = 0x00000007
        vlan_mode_support= 0x05
        vlan_filter_cnt  = 8
        mixed_filter_cnt = 2
        mcast_filter_cnt = 0
        ucast_filter_cnt = 0
        channel_cnt      = 1
```

## Get NC-SI Pass-through Statistics (0x1A)

```
# ncsi-tool --get_pass_thru_stats

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
Get NCSI Pass-through Statistics
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
NC-SI Pass-through Statistics :
        TX Packets: 0x00000000000001b1
        TX Dropped: 0x00000000
        RX Packets: 0x00000365
        RX Dropped: 0x00000000
```

## 3.5 Raw Command Example

```
# ncsi-tool --raw "XX YY ZZ ..." --ctrl_packet_type 0xAA
```

### Example 1: Send control packet type 0x50 with appropriate raw command to Get Host's MAC address (System MAC Address).

In this example, we use ncsi-tool to send an OEM command to the Intel I210.

```
# ncsi-tool --raw "00 00 01 57 06" --ctrl_packet_type 0x50

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
--- response data shown byte-by-byte length[11] ---
[0000 - 0003]    00 00 01 57
[0004 - 0007]    06 A4 BF 01
[0008 - 0010]    35 F2 48
```

The image below shows the I210's OEM command for retrieving the Host's MAC address (System MAC Address).

For more detail, please refer to [**Intel® Ethernet Controller I210 Datasheet, Revision Number: 3.7**]

---

*Ethernet Controller I210 —System Manageability*

## 10.6.3.9 System MAC Address

### 10.6.3.9.1 Get System MAC Address Command (Intel Command 0x06)

In order to support a system configuration that requires the NC to hold the MAC address for the MC (such as shared MAC address mode), the following command is provided to enable the MC to query the NC for a valid MAC address.

The NC must return the system MAC addresses. The MC should use the returned MAC addressing as a shared MAC address by setting it using the Set MAC Address command as defined in NC-SI 1.0.

It is also recommended that the MC use packet reduction and Manageability-to-Host command to set the proper filtering method.

| | Bits | | | |
|---|---|---|---|---|
| Bytes | 31:24 | 23:16 | 15:08 | 07:00 |
| 00...15 | NC-SI Header | | | |
| 16...19 | Manufacturer ID (Intel 0x157) | | | |
| 20 | 0x06 | | | |

### 10.6.3.9.2 Get System MAC Address Response (Intel Command 0x06)

| | Bits | | | |
|---|---|---|---|---|
| Bytes | 31:24 | 23:16 | 15:08 | 07:00 |
| 00...15 | NC-SI Header | | | |
| 16...19 | Response Code | | Reason Code | |
| 20...23 | Manufacturer ID (Intel 0x157) | | | |
| 24...27 | 0x06 | MAC Address | | |
| 28...30 | MAC Address | | | |

## Example 2: Send NC-SI raw command to get channel capabilities.

∗ Its response data is the same as "ncsi-tool --get_capabilities".

∗ --package & --channel options are optional. Package ID and channel ID will be default as #0 if both options are not specified in the command line.

```
# ncsi-tool --raw "" --ctrl_packet_type 0x16 --package 0 --channel 0
Version : 2.0
NCSI-0 chan(00:00)
- send from ncsi0
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
--- response data shown byte-by-byte length[28] ---
[0000 - 0003]    00 00 00 1F
[0004 - 0007]    00 00 00 0F
[0008 - 0011]    00 00 00 07
[0012 - 0015]    00 00 1F F0
[0016 - 0019]    00 00 00 07
[0020 - 0023]    08 02 00 00
[0024 - 0027]    00 00 05 01
```

**Table 88 – Get Capabilities command packet format**

| | Bits | | | |
|---|---|---|---|---|
| **Bytes** | **31..24** | **23..16** | **15..08** | **07..00** |
| **00..15** | NC-SI Control Packet Header | | | |
| **16..19** | Checksum | | | |
| **20..45** | Pad | | | |

**Table 89 – Get Capabilities response packet format**

| | Bits | | | |
|---|---|---|---|---|
| **Bytes** | **31..24** | **23..16** | **15..08** | **07..00** |
| **00..15** | NC-SI Control Packet Header | | | |
| **16..19** | Response Code | | Reason Code | |
| **20..23** | Capabilities Flags | | | |
| **24..27** | Broadcast Packet Filter Capabilities | | | |

| | Bits | | | |
|---|---|---|---|---|
| **Bytes** | **31..24** | **23..16** | **15..08** | **07..00** |
| **28..31** | Multicast Packet Filter Capabilities | | | |
| **32..35** | Buffering Capability | | | |
| **36..39** | AEN Control Support | | | |
| **40..43** | VLAN Filter Count | Mixed Filter Count | Multicast Filter Count | Unicast Filter Count |
| **44..47** | Reserved | | VLAN Mode Support | Channel Count |
| **48..51** | Checksum | | | |

# 4 ncsi_test User Guide

## 4.1 Introduction

∗ ncsi_test is a tool for verifying the HAL NCSI-releate APIs of ipmid.

∗ ncsi_test uses the HAL NCSI-releate APIs of ipmid to communicate with the Linux NCSI stack.

∗ When you need to test the HAL NCSI-releate APIs of ipmid, use this tool.

∗ If you want to send NC-SI commands, please refer to ncsi-tool.

## 4.2 Command Usage

```
ncsi_test <nic_idx> <package_id> <channel_id>

 - Test NCSI command with (nic_idx, package_id, channel_id) = (1, 0, 1)
   - e.g. ./ncsi_test 1 0 1
```

## 4.3 Example

```
root@amd-obmc:~# ncsi_test 0 0 0

Copyright (c) 2013-2023, Insyde Software Corp. All Rights Reserved.

This is a tool to verify NCSI over RMII
- (nic_idx, package_id, channel_id) = (0, 0, 0)

==================================================

Show pacakge_id and how many NCSI channels in nic_idx[0]
[0][0/1] (package_id, channel_counts) = (0, 1)
NCSI[0:0:0] -> Send Clear Initial State
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]

==================================================

NCSI[0:0:0] -> send Get Version ID
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
NC-SI Version : 1.0.0
Firmware Name : NX1NCSI
Firmware Version : 01.05.28.00
PCI DID : 0x165F
PCI VID : 0x14E4
PCI SSID : 0x165F
PCI SVID : 0x14E4
Manufacturer ID : 0x113D

==================================================

NCSI[0:0:0] -> send Get Controller Packet Statistics [0x18]
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
i[0] Total Bytes Received = 57906
i[1] Total Bytes Transmitted = 11506
i[2] Total Unicast Packets Received = 49
i[3] Total Multicast Packets Received = 226
i[4] Total Broadcast Packets Received = 71
i[5] Total Unicast Packets Transmitted = 31
i[6] Total Multicast Packets Transmitted = 1
i[7] Total Broadcast Packets Transmitted = 0
i[8] FCS Received Errors = 0
i[9] Alignment Errors = 0
i[10] False Carrier Detections = Not Supported
i[11] Runt Packets Received = 0
i[12] Jabber Packets Received = 0
i[13] Pause XON Frames Received = 0
i[14] Pause XOFF Frames Received = 0
i[15] Pause XON Frames Transmitted = 0
i[16] Pause XOFF Frames Transmitted = 0
i[17] Single Collision Transmit Frames = 0
i[18] Multiple Collision Transmit Frames = 0
i[19] Late Collision Frames = 0
i[20] Excessive Collision Frames = 0
i[21] Control Frames Received = 0
i[22] 64 Byte Frames Received = Not Supported
i[23] 65-127 Byte Frames Received = Not Supported
i[24] 128-255 Byte Frames Received = Not Supported
i[25] 256-511 Byte Frames Received = Not Supported
```

```
i[26] 512-1023 Byte Frames Received = Not Supported
i[27] 1024-1522 Byte Frames Received = Not Supported
i[28] 1523-9022 Byte Frames Received = Not Supported
i[29] 64 Byte Frames Transmitted = Not Supported
i[30] 65-127 Byte Frames Transmitted = Not Supported
i[31] 128-255 Byte Frames Transmitted = Not Supported
i[32] 256-511 Byte Frames Transmitted = Not Supported
i[33] 512-1023 Byte Frames Transmitted = Not Supported
i[34] 1024-1522 Byte Frames Transmitted = Not Supported
i[35] 1523-9022 Byte Frames Transmitted = 0
i[36] Valid Bytes Received = 57906
i[37] Error Runt Packets Received = Not Supported
i[38] Error Jabber Packets Received = Not Supported

==================================================

NCSI[0:0:0] -> send Get NCSI Statistics [0x19]
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
i[0] NC-SI Command Received = 3420
i[1] NC-SI Control Packets Dropped = 0
i[2] NC-SI Command Type Errors = 2
i[3] NC-SI Command Checksum Errors = 0
i[4] NC-SI Receive Packets = 3423
i[5] NC-SI Transmit Packets = 3422
i[6] NC-SI AENs Sent = 0

==================================================

NCSI[0:0:0] -> send Get Pass-Through Statistics [0x1A]
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
Pass-through TX Packets Received On NC-SI Interface : 7159
i[0] Pass-through TX Packets Dropped = 0
i[1] Pass-through TX Packet Channel State Errors = 0
i[2] Pass-through TX Packet Undersized Errors = 0
i[3] Pass-through TX Packet Oversized Errors = 0
i[4] Pass-through RX Packet Received on LAN Interface = 42967
i[5] Total Pass-through RX Packets Dropped = Not Supported
i[6] Pass-through RX Packet Channel State Errors = Not Supported
i[7] Pass-through RX Packet Undersized Errors = Not Supported
i[8] Pass-through RX Packet Oversized Errors = Not Supported

==================================================

NCSI[0:0:0] : Send raw command
response_code   = 0x0 [Successful]
reason_code     = 0x0 [No Error]
--- response data shown byte-by-byte length[28] ---
[0000 - 0003]    00 00 00 5F
[0004 - 0007]    00 00 00 0F
[0008 - 0011]    00 00 00 07
[0012 - 0015]    00 00 40 00
[0016 - 0019]    00 00 00 07
[0020 - 0023]    02 08 00 00
[0024 - 0027]    00 00 07 02

==================================================

NCSI[0:0:0] : Get Temperature Command - Broadcom Only
Get On-Chip Temperature
response_code   = 0x3 [Unsupported]
```

```
reason_code      = 0x7fff [Unknown Command Type]
There is no response data.
Current Temperature is 0 Celsius
Maximum Temperature is 0 Celsius
Get Optical Module Temperature
response_code    = 0x3 [Unsupported]
reason_code      = 0x7fff [Unknown Command Type]
There is no response data.
Current Temperature is 0 Celsius
Maximum Temperature is 0 Celsius

==================================================

Get Host MAC address from NCSI OEM command
NCSI[0:0:0] -> mac[00:10:18:00:00:00]

==================================================

test all done
```

When the response code and reason code are both zero and the Host MAC address is displayed, it means everything is working properly.

```
response_code = 0x0 [Successful]
reason_code = 0x0 [No Error]

NCSI[0:0:0] -> mac[00:10:18:00:00:00]
```

# 5 AST2600 H/W Crypto User Guide

## 5.1 Introduction

AST2600 kernel driver provides a hardware SoC solution to handle the task of crypto. e.g. Crypto Driver. For more info about how it is ported in your BMC, please refer to **Ch21 H/W Crypto User Guide** in **Supervyse OPF Platform Porting Guide**.

## 5.2 User Space Test Utilities

Two tools are provided to verify AST2600 hardware crypto feature.

**hwcrypto_test**

As shown below,  plain_text named "-Supvervyse OPF-X" is encrypted. Then this encrypt_test is decrypted and shown on the BMC console.

1. To decrypt encrypt_text, and show decrypt_text on BMC console.
2. As we can see below, the plain_text and decrypto_text are the same.

```
root@intel-obmc:~# hwcrypto_test

Copyright (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

test crypto with cbc(aes) & skcipher

plain_text : -Supervyse OPF-X
encrypt...
encrypt_text :

        bf ba 74 29
        36 07 a2 28
        8b 6d 09 38
        1c a3 bb 13

decrypt...
decrypt_text : -Supervyse OPF-X
root@intel-obmc:~#
```

**hwcrypto.sh**

A tool to verify ast2600 hardware crypto driver by using openssl.

```
root@intel-obmc:~# hwcrypto.sh
Testing aes-256-cbc Encrypt/Decrypt
openssl aes-256-cbc -nosalt -K "0123456789abcdef0123456789abcdef" -engine afalg -iv
"12345678901234561234567890123456" -e -in /tmp/plain_text -out /tmp/aes-256-cbc.enc
engine "afalg" set.
hex string is too short, padding with zero bytes to length
openssl aes-256-cbc -nosalt -K "0123456789abcdef0123456789abcdef" -engine afalg -iv
"12345678901234561234567890123456" -d -in /tmp/aes-256-cbc.enc -out /tmp/aes-256-
cbc.dec
```

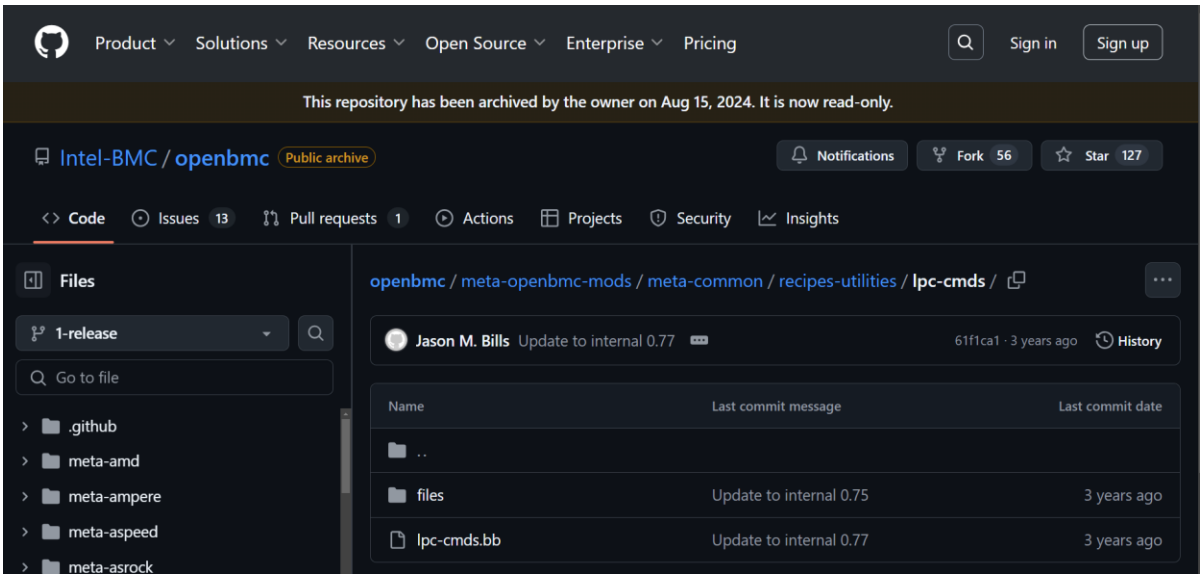
```
engine "afalg" set.
hex string is too short, padding with zero bytes to length
cmp /tmp/plain_text /tmp/aes-256-cbc.dec
aes-256-cbc encrypt/decrypt : PASS

root@intel-obmc:/nv#
```

# 6 lpc_cmds User Guide

## 6.1 Introduction

**lpc_cmds** is a verification tool that supports Intel, AMD, and NVIDIA platforms to verify **KCS**, **Snoop**, **SIO** and **Mailbox** interfaces between BMC and Host. This tool was originally released by Intel (https://github.com/Intel-BMC/openbmc/tree/1-release/meta-openbmc-mods/meta-common/recipes-utilities/lpc-cmds).




## 6.2 KCS Driver

There are total 4 KCS devices in AST2600 eSPI/LPC controller for data transaction between BMC and Host. e.g. **/dev/ipmi-kcs1, /dev/ipmi-kcs2, /dev/ipmi-kcs3, /dev/ipmi-kcs4**.

- /dev/ipmi-kcs1 : Not in use
- /dev/ipmi-kcs2 : Not in use
- /dev/ipmi-kcs3 : IPMI Host Interface
- /dev/ipmi-kcs4 : MCTP over KCS

**Prerequisites**

**/dev/ipmi-kcs[1...4]** are usually bound by systemd service **phosphor-ipmi-kcs@ipmi_kcs.** Before using lpc_cmds to test KCS driver, you have to stop **phosphor-ipmi-kcs@ipmi_kcs**.

```
# systemctl stop phosphor-ipmi-kcs@ipmi_kcs3
```

**Command Usage**

***lpc_cmds*** are used to verify the data transaction between Host and BMC.

```
# lpc_cmds kcs [1 ~ 4]

# lpc_cmds kcs [1 ~ 4] quiet
```

- "kcs 1" is for */dev/ipmi-kcs1*
- "kcs 2" is for */dev/ipmi-kcs2*
- "kcs 3" is for */dev/ipmi-kcs3*
- "kcs 4" is for */dev/ipmi-kcs4*

**Example I**

At BMC console, issue the following command to handle data transaction between Host and BMC.

```
# lpc_cmds kcs 3
```

At Host side (UEFI shell), issue the following **Get Device ID** command

```
/> ipmitool 20 18 1
```

And the following messages are printed at BMC console.

```
KCS Request >>>>>>>>>>>>>>>>>>>>>>>>>>
18   01
=====================================

KCS Response <<<<<<<<<<<<<<<<<<<<<<<<<
1C 01 C1
=====================================
```

**Example II**

At BMC console, issue the following command to handle data transaction between Host and BMC. Issue with Parameter **quiet** means the quit mode. In other words, no further debug message is shown at BMC side.

```
# lpc_cmds kcs 3 quiet
```

At Host side (UEFI shell), issue the following **Get Device ID** command

```
/> ipmitool 20 18 1
```

At BMC console, there is no message shown. And KCS Response is handled by ***lpc_cmds*** quietly.

## 6.3 Snoop Driver

There are 2 Snoop devices in the AST2600 eSPI/LPC controller for capturing BIOS POST codes. For example, **/dev/aspeed-lpc-snoop0** and **/dev/aspeed-lpc-snoop1**.

/dev/aspeed-lpc-snoop0: Default I/O base address is 0x80

/dev/aspeed-lpc-snoop1: Disabled by default

**Prerequisites**

**/dev/aspeed-lpc-snoop** are usually bound by systemd service **lpcsnoop**. Before using lpc_cmds to test Snoop driver, you have to stop **lpcsnoop**.

```
# systemctl stop lpcsnoop
```

**Command Usage**

`lpc_cmds` is used to verify the captured POST codes.

```
# lpc_cmds snoop 0 read
# lpc_cmds snoop 1 read
```

**Example I**

At the BMC console, issue the following command to read the BIOS POST codes captured by Snoop 0:

```
# lpc_cmds snoop 0 read
BIOS POST Codes in Hex (35 entries):
 0: 02
 1: 03
 2: 03
 3: 04
 4: 04
 5: 05
 6: 05
 7: 06
 8: 70
 9: 74
 10: 76
 11: A1
 12: A3
 13: A3
 14: A3
 15: A3
 16: A3
 17: A3
 18: A7
 19: A9
 20: A7
 21: A7
 22: A7
 23: A7
 24: A8
 25: A9
 26: A9
 27: AA
```

```
28: AE
29: AF
30: E0
31: E1
32: E4
33: E3
34: E1
```

## 6.4 SIO Driver

The AST2600 eSPI/LPC controller provides system control and monitoring functions through **/dev/lpc-sio**. It supports various ACPI, power control, and system management functions.

**Command Usage**

```
# lpc_cmds sio <COMMAND>
```

| <COMMAND> | Description |
|---|---|
| **get_acpi_state** | To get the state of *ACPI SLP*. |
| **get_acpi_changed** | To get the state of *ACPI SLP* and check whether the stats is changed or not. |
| **get_pwrgd_status** | To get the status of *Power Good*. |
| **get_pwrgd_changed** | To get the state of *Power Good* and check whether the stats is changed or not. |
| **get_onctl_status** | To get the state of *On Control*. |
| **set_onctl_gpio_disable** | To disable GPIO mode on *ONCTL*. |
| **set_onctl_gpio_high** | To enable GPIO mode on ONCTL.<br>To enable GPIO output on ONCTL and set GPIO level high on ONCTL. |
| **set_onctl_gpio_low** | To enable GPIO on ONCTL and set GPIO level low on ONCTL. |
| **get_pwrbtn_override_status** | To read the status of *Power Button Override*. |
| **get_pwrbtn_override_status_clear** | To read the status of *Power Button Override*, and then write 1 to clear if it is on. |
| **get_pfail_status** | To the status of pfail. |
| **set_bmc_sci_event** | To enable SCI event enable, enable wake-up event and trigger wake-up event. |
| **clear_bmc_sci_event** | To clear the status of wake-up event and the status of SCI event. |
| **set_bmc_smi_event** | To enable wake-up event, enable SMI interrupt as BMC trigger wake-up event and trigger wake-up event. |

| | |
|---|---|
| **clear_bmc_smi_event** | To clear the status of wake-up event. |

**Example I**

At the BMC console, issue the following command to get ACPI state

```
# lpc_cmds sio get_acpi_state
ACPI SLP state --> SLP_45
```

- To get the state of ACPI SLP.

## 6.5 Mailbox Driver

The AST2600 eSPI/LPC mailbox controller contains 32 mailbox registers for data transactions between BMC and Host. Each register supports 1-byte data configuration.

**Prerequisites**

**/dev/aspeed-mbox** are usually bound by systemd service "**xyz.openbmc_project.Host.Misc.Manager"**. Before using lpc_cmds to test Mailbox driver, you have to stop **xyz.openbmc_project.Host.Misc.Manager**.

```
# systemctl stop xyz.openbmc_project.Host.Misc.Manager
```

**Command Usage**

The *lpc_cmds mailbox* commands are used to read from and write to mailbox registers:

```
# lpc_cmds mailbox read  (0 ~ 31)

# lpc_cmds mailbox write (0 ~ 31) (0x00 ~ 0xFF)
```

**Example I**

First, read the initial value of mailbox register 3:

```
# lpc_cmds mailbox read 3
MailBox3 read value : 0x00
```

Write a new value (0x87) to mailbox register 3:

```
# lpc_cmds mailbox write 3 87
MailBox3 write value : 0x87 done!
```

Read mailbox register 3 again to verify the value has changed:

```
# lpc_cmds mailbox read 3
MailBox3 read value : 0x87
```

# 7 Register Access Tool User Guide

**Register Access Tool**, aka ***io*** is a user space tool (Intel proprietary) to check CPU info(e.g. AST2600, AST2700,...etc) and access AST2600 registers for the purpose of debugging. Similar tools are *md* and *mw* in u-boot and *devmem* in user space.

```
/usr/sbin/io
```

## 7.1 AST2600 I/O Map

| Physical Address | Address Length |
|---|---|
| 0x0 | 0x10000000 |
| 0x1e600000 | 0x200000 |
| 0x20000000 | 0x10000000 |
| 0x30000000 | 0x10000000 |
| 0x40000000 | 0x10000000 |
| 0x50000000 | 0x60000000 |

- When typing *io* in BMC shell, the following info is shown.
- When *io* is invoked, a shell called "io shell" is shown.
- The data after "added map:" is the newly mapped virtual address.

```
root@bmc-mac7648feec094b:~# io
Found AST2600
added map: 0x66d8b000 (0..10000000)
added map: 0x66b8b000 (1e600000..1e800000)
added map: 0x56b8b000 (20000000..30000000)
added map: 0x46b8b000 (30000000..40000000)
added map: 0x36b8b000 (50000000..60000000)
> ?
md addr [len]
mw addr val [len]
id [0-2]
q[uit] | ctrl-d | ctrl-c to exit
> q
root@bmc-mac7648feec094b:~#
```

## 7.2 io shell

After entering io shell prompt ">", type the following commands:

```
md addr [len]
mw addr val [len]
q[uit] | ctrl-d | ctrl-c to exit
(addr, val, len are all hex numbers)
```

```
id [0-2]:
  0: read manufacturer id of firmware spi flash
  1: read manufacturer id of spi1 flash
  2: read manufacturer id of spi2 flash
```

**AST2600 Register Reading/Writing**

- **md**: To read AST2600 registers.
- **mw**: To write AST2600 registers.

Here is the example to read 0x1E78401C and wirte 0x1E78401C with 1 byte data 0x10, where 0x1E78401C is UART_SCR: Scratch Register for UART5 controller. For more information, please refer to AST2600 datasheet. e.g. AST2600 Integrated Remote Management Processor A3 Datasheet, Version 1.4.

```
root@bmc-mac7648feec094b:~# io
Found AST2600
added map: 0x66df0000 (0..10000000)
added map: 0x66bf0000 (1e600000..1e800000)
added map: 0x56bf0000 (20000000..30000000)
added map: 0x46bf0000 (30000000..40000000)
added map: 0x36bf0000 (50000000..60000000)
> md 1e78401c 1
1e78401c: 00000000                                ....
> mw 1e78401c 0x10 1
> md 1e78401c 1
1e78401c: 00000010                                ....
> mw 1e78401c 0x0 1
> md 1e78401c 1
1e78401c: 00000000                                ....
> q
```

**The Reading of SPI Manufacturer ID**

Here is the example to read the SPI Manufacturer ID from Firmware SPI flash and SPI Flash 1. The

reading is done by direct access to the address of Firmware SPI Controller and SPI1 Controller, by using JDEC READ ID opcode (0x9F). For more information, please refer to JEDEC document.

```
root@bmc-mac7648feec094b:~# io
Found AST2600
added map: 0x66e27000 (0..10000000)
added map: 0x66c27000 (1e600000..1e800000)
added map: 0x56c27000 (20000000..30000000)
added map: 0x46c27000 (30000000..40000000)
added map: 0x36c27000 (50000000..60000000)
> id 0
SPI ID: 0x1b 0x20 0xc2 (Manufacturer ID: 0xc2)
> id 1
SPI ID: 0x19 0x40 0xef (Manufacturer ID: 0xef)
> q
root@bmc-mac7648feec094b:~#
```

# 8 SPI Tool User Guide

The mtd tools we use to test SPI is from https://git.infradead.org/mtd-utils.git. e.g. *mtd_debug*, *mtdinfo*. MTD stands for **M**emory **T**echnology **D**evices and means flash, RAM and similar chips, used for solid state storage on embedded devices.

```
root@bmc-mac7648feec094b:~# cat /proc/mtd
dev:    size   erasesize  name
mtd0: 08000000 00010000 "bmc"
mtd1: 00010000 00010000 "spl"
mtd2: 000f0000 00010000 "u-boot"
mtd3: 07600000 00010000 "image-a"
mtd4: 00200000 00010000 "sofs"
mtd5: 006c0000 00010000 "rwfs"
mtd6: 00010000 00010000 "certificate"
mtd7: 00010000 00010000 "sdr"
mtd8: 00010000 00010000 "u-boot-env"
mtd9: 00010000 00010000 "sdk-info"
```

## 8.1 mtd_debug

mtd_debug is a tool not only to show the information of mtd device, but also read, write and erase a mtd device.

```
root@bmc-mac7648feec094b:~# mtd_debug
usage: mtd_debug info <device>
       mtd_debug read <device> <offset> <len> <dest-filename>
       mtd_debug write <device> <offset> <len> <source-filename>
       mtd_debug erase <device> <offset> <len>
root@bmc-mac7648feec094b:~#
```

**Example 1**

```
root@bmc-mac7648feec094b:~# mtd_debug info /dev/mtd0
mtd.type = MTD_NORFLASH
mtd.flags = MTD_CAP_NORFLASH
mtd.size = 134217728 (128M)
mtd.erasesize = 65536 (64K)
mtd.writesize = 1
mtd.oobsize = 0
regions = 0

root@bmc-mac7648feec094b:~#
```

**Example 2**

```
root@bmc-mac7648feec094b:~# mtd_debug read /dev/mtd/sdk-info 0x0 0x10 /tmp/data
Copied 16 bytes from address 0x00000000 in flash to /tmp/data
root@bmc-mac7648feec094b:~#
```

## 8.2 mtdinfo

mtdinfo is a tool to show MTD information.

```
root@bmc-mac7648feec094b:~# mtdinfo --help
mtdinfo version 2.1.5 - a tool to print MTD information.
```

```
Usage: mtdinfo <MTD node file path> [--map | -M] [--ubi-info | -u]
       mtdinfo --all [--ubi-info | -u]
       mtdinfo [--help | --version]

Options:
-u, --ubi-info                  print what would UBI layout be if it was put
                                on this MTD device
-M, --map                       print eraseblock map
-a, --all                       print information about all MTD devices
                                Note: `--all' may give less info per device
                                than, e.g., `mtdinfo /dev/mtdX'
-h, --help                      print help message
-V, --version                   print program version

Examples:
  mtdinfo /dev/mtd0             print information MTD device /dev/mtd0
  mtdinfo /dev/mtd0 -u          print information MTD device /dev/mtd0
                                and include UBI layout information
  mtdinfo -a                    print information about all MTD devices
```

**Example 1**

```
root@bmc-mac7648feec094b:~# mtdinfo
Count of MTD devices:           10
Present MTD devices:            mtd0, mtd1, mtd2, mtd3, mtd4, mtd5, mtd6, mtd7, mtd8,
mtd9
Sysfs interface supported:      yes
root@bmc-mac7648feec094b:~#
```

**Example 2**

```
root@bmc-mac7648feec094b:~# mtdinfo /dev/mtd0
mtd0
Name:                           bmc
Type:                           nor
Eraseblock size:                65536 bytes, 64.0 KiB
Amount of eraseblocks:          2048 (134217728 bytes, 128.0 MiB)
Minimum input/output unit size: 1 byte
Sub-page size:                  1 byte
Character device major/minor:   90:0
Bad blocks are allowed:         false
Device is writable:             true

root@bmc-mac7648feec094b:~#
```

# 9 pwm_tool User Guide

## 9.1 Introduction

pwm_tool is a tool to verify the PWM/FAN devices.

PWM (Pulse Width Modulation) is a common signal control method. It adjusts the pulse width of a digital signal to control the power or behavior of analog devices. It is often used to control motor speed, LED brightness, or create sound signals.

- Basic Principle of PWM

PWM works by outputting a square wave signal with a fixed frequency. By changing the time the signal stays high (pulse width) compared to the total period, it controls the output.

- Square Wave Signal
  1. A square wave has two states: **high** (ON) and **low** (OFF).
  - **Period (T):** The time it takes for one full cycle of high and low.
  - **Frequency (f):** The number of cycles per second, calculated as: *f=1Tf= \frac{1}{T}*f=T1
- Duty Cycle
  Duty cycle is the percentage of time the signal stays high in one period:

Examples:

- **50% duty cycle:** High and low times are equal.
- **25% duty cycle:** High time is one-fourth of the period.
- **75% duty cycle:** High time is three-fourths of the period.

PWM on BMC can be divided into internal and external two types, internal means the use of the AST2600 on-chip PWM pin, external means the use of external chip PWM function. As long as they are implemented through hwmon, pwm_tool can be used.

## 9.2 Command Usage

The pwm_tool can be found in the /usr/bin filesystem.

```
root@bmc-mac0007e934a4c6:/usr/bin# pwm_tool

Copyright (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

Usage :
        pwm_tool test_pwm [HWMON number-PWM number] [HWMON number-TACH number] [max fan rpm] [error] [wait time]
                 [HWMON number-PWM number]   :The "HWMON number" is the hwmon number of the pwm positions.
                                              The PWM number is the pwm number.
                                              You can check the "/sys/class/hwmon/" and find them.
                                              ex:0-1 means the pwm path /sys/class/hwmon/hwmon0/pwm1

                 [HWMON number-TACH number]   :The "HWMON number" is the hwmon number of the tach positions.
                                              The TACH number is the tach number.
                                              You can check the "/sys/class/hwmon/" and find them.
                                              ex:0-1 means the tach path /sys/class/hwmon/hwmon0/fan1_input

                 [max fan rpm]  : maximum fan speed

                 [error]        : Tolerance 5~10 (%)

                 [wait time]    : test wait time(s)
```

HWMON number-PWM number: Location of the PWM device.

HWMON number-TACH number: Location of the TACH device.

Max fan rpm: Maximum fan rpm.

Error: Allowable error in the test, usually 8%.

Wait time: Time between each fan PWM adjustment.

Assuming your PWM devices are in hwmon3 and hwmon4.

```
hwmon3:
device        fan1_input    fan2_fault    fan3_enable   fan3_target   fan4_input
fan5_fault    fan6_enable   fan6_target   pwm1          pwm2_enable   pwm4
pwm5_enable   subsystem
fan1_enable   fan1_target   fan2_input    fan3_fault    fan4_enable   fan4_target
fan5_input    fan6_fault    name          pwm1_enable   pwm3          pwm4_enable   pwm6
uevent
fan1_fault    fan2_enable   fan2_target   fan3_input    fan4_fault    fan5_enable
fan5_target   fan6_input    of_node       pwm2          pwm3_enable   pwm5
pwm6_enable

hwmon4:
device        fan1_input    fan2_fault    fan3_enable   fan3_target   fan4_input
fan5_fault    fan6_enable   fan6_target   pwm1          pwm2_enable   pwm4
pwm5_enable   subsystem
fan1_enable   fan1_target   fan2_input    fan3_fault    fan4_enable   fan4_target
fan5_input    fan6_fault    name          pwm1_enable   pwm3          pwm4_enable   pwm6
uevent
fan1_fault    fan2_enable   fan2_target   fan3_input    fan4_fault    fan5_enable
fan5_target   fan6_input    of_node       pwm2          pwm3_enable   pwm5
pwm6_enable
```

The fan input rpm corresponding to hwmon**3**/pwm**1** is hwmon**3**/fan**1**_input, and we use the instruction:

```
pwm_tool test_pwm 3-1 3-1 11000 5 5
```

The results of the test are:

```
root@bmc-mac0007e9345b24:/sys/class/hwmon/hwmon3# pwm_tool test_pwm 3-1 3-1 11000 5 5

Copyright (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

================================
Test PWM /sys/class/hwmon/hwmon3/pwm1 and TACH /sys/class/hwmon/hwmon3/fan1_input:
pwm: /sys/class/hwmon/hwmon3/pwm1 ,write 0 and read 0,  test pass.
pwm: /sys/class/hwmon/hwmon3/pwm1 ,write 127 and read 127,  test pass.
pwm: /sys/class/hwmon/hwmon3/pwm1 ,write 255 and read 255,  test pass.
wait 5 seconds to read TACH.
Test PWM value = 255, read TACH value = 10685
Reasonable fan speed range is [11550 ~ 10450]
pwm: /sys/class/hwmon/hwmon3/pwm1 , test TACH pass.
PWM TACH Success!
--------------------------
```

# 10 mdio-tool User Guide

## 10.1 Introduction

The mdio-tool command is used for reading from or writing to MDIO (Management Data Input/Output) devices.

## 10.2 Command Usage

```
# mdio-tool
Usage mii [r/w] [dev] [reg] [val]
```

**r**: Read from the device.

**w**: Write to the device.

**dev**: The LAN interface name.

**reg**: The register within the device to read from or write to.

**val** (optional): The value to write to the specified register (required only for write operations).

## 10.3 Example

Read the PHY Identifier Register of the PHY.

The value depends on your PHY model and can be found in the PHY datasheet.

```
root@evb-ast2600-insyde:~# mdio-tool r eth0 2
0x0362

root@evb-ast2600-insyde:~# mdio-tool r eth0 3
0x5d12

root@evb-ast2600-insyde:~# mdio-tool r eth1 2
0x001c

root@evb-ast2600-insyde:~# mdio-tool r eth1 3
0xc916
```

For more details about the PHY Identifier Register, please refer to [**IEEE 802.3-2018 - 22.2.4.3.1 PHY Identifier (Registers 2 and 3)**].

# 11 iperf3 User Guide

## 11.1 Introduction

iperf3 is a tool for performing network throughput measurements. It can test TCP, UDP, or SCTP throughput.

To perform an iperf3 test the user must establish both a server and a client.

For more detail, please refer to https://manpages.org/iperf3.

## 11.2 Example

Start an iperf3 server on the BMC console using the following command:

```
root@bmc-macd2ed08e915d0:~# iperf3 -s
-----------------------------------------------------------
Server listening on 5201 (test #1)
-----------------------------------------------------------
```

Then, run iperf3 in client mode from another PC:

**To test BMC RX (receive):**

```
$ iperf3 -c 192.168.21.246
Connecting to host 192.168.21.246, port 5201
[  5] local 172.17.174.255 port 38906 connected to 192.168.21.246 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  11.8 MBytes  98.8 Mbits/sec    5   72.1 KBytes
[  5]   1.00-2.00   sec  11.0 MBytes  92.7 Mbits/sec    2    103 KBytes
[  5]   2.00-3.00   sec  11.2 MBytes  93.6 Mbits/sec    7   97.6 KBytes
[  5]   3.00-4.00   sec  11.4 MBytes  95.6 Mbits/sec    8   84.8 KBytes
[  5]   4.00-5.00   sec  11.2 MBytes  93.6 Mbits/sec    2    103 KBytes
[  5]   5.00-6.00   sec  11.2 MBytes  94.1 Mbits/sec    4    100 KBytes
[  5]   6.00-7.00   sec  11.1 MBytes  93.2 Mbits/sec    5   97.6 KBytes
[  5]   7.00-8.00   sec  11.2 MBytes  93.6 Mbits/sec    8   97.6 KBytes
[  5]   8.00-9.00   sec  11.2 MBytes  94.1 Mbits/sec    5   97.6 KBytes
[  5]   9.00-10.00  sec  9.20 MBytes  77.2 Mbits/sec    3   80.6 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec   110 MBytes  92.7 Mbits/sec   49             sender
[  5]   0.00-10.01  sec   110 MBytes  92.1 Mbits/sec                  receiver

iperf Done.
```

**To test BMC TX (transmit):**

```
$ iperf3 -c 192.168.21.246 -R
Connecting to host 192.168.21.246, port 5201
Reverse mode, remote host 192.168.21.246 is sending
[  5] local 172.17.174.255 port 53036 connected to 192.168.21.246 port 5201
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-1.00   sec  11.0 MBytes  92.3 Mbits/sec
[  5]   1.00-2.00   sec  11.2 MBytes  93.7 Mbits/sec
[  5]   2.00-3.00   sec  11.0 MBytes  92.0 Mbits/sec
[  5]   3.00-4.00   sec  11.2 MBytes  93.9 Mbits/sec
[  5]   4.00-5.00   sec  11.0 MBytes  92.3 Mbits/sec
[  5]   5.00-6.00   sec  11.2 MBytes  94.0 Mbits/sec
[  5]   6.00-7.00   sec  11.2 MBytes  93.9 Mbits/sec
[  5]   7.00-8.00   sec  11.1 MBytes  92.9 Mbits/sec
[  5]   8.00-9.00   sec  10.9 MBytes  91.3 Mbits/sec
[  5]   9.00-10.00  sec  11.2 MBytes  94.0 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec   112 MBytes  93.8 Mbits/sec    0             sender
[  5]   0.00-10.00  sec   111 MBytes  93.0 Mbits/sec                  receiver

iperf Done.
```

# 12 eth-util User Guide

## 12.1 Introduction

eth-util is a tool for checking LAN device information. (e,g PHY ID)

## 12.2 Command Usage

```
# eth <LAN interface name>
```

## 12.3 Example

```
root@evb-ast2600-insyde:~# eth eth0

Portions Copyright (C) 2023 Insyde Software Corp.

Ethtool Device: eth0
MAC: 26:7d:51:5b:1b:54
Speed: 100Mbps; full-Duplex; Link: UP
phy_id[0x3625D12][Broadcom BCM54616S]
```

# 13 hwclock User Guide

## 13.1 Introduction

- hwclock can read and write the time for the ASPEED RTC driver
- The source code of hwclock is in BusyBox

## 13.2 Command Usage

- hwclock command
    - -s: Set the System Time from the Hardware Clock
    - -w: Set the Hardware Clock to the current System Time
    - -u: Assume the Hardware Clock is kept in UTC
    - -l: Assume the Hardware Clock is kept in local time
    - --systz: Set the System Time based on the current timezone
    - -f: Specify the RTC device file to use, instead of the default /dev/rtc

```
hwclock [-swul] [--systz] [-f DEV]
```

## 13.3 Usage examples and precautions

- When initially testing hwclock, it is recommended to stop the time sync functionality first
    - systemctl stop systemd-timesyncd.service
- Use the following commands to verify that the RTC time can be read and written

```
systemctl stop systemd-timesyncd.service

//read the RTC time
hwclock

//write the system time to RTC
hwclock -w
```

```
root@amd-obmc:~# systemctl stop systemd-timesyncd.service
root@amd-obmc:~#
root@amd-obmc:~#
root@amd-obmc:~# date
Tue Jul 16 07:36:43 UTC 2024
root@amd-obmc:~#
root@amd-obmc:~# date -s "2021-02-19 14:51:30"
Fri Feb 19 14:51:30 UTC 2021
root@amd-obmc:~#
root@amd-obmc:~# date
Fri Feb 19 14:51:32 UTC 2021
root@amd-obmc:~#
root@amd-obmc:~#
root@amd-obmc:~# hwclock
Tue Jul 16 07:36:51 2024  0.000000 seconds
root@amd-obmc:~#
root@amd-obmc:~# hwclock -w
root@amd-obmc:~#
root@amd-obmc:~# hwclock
Fri Feb 19 14:51:39 2021  0.000000 seconds
root@amd-obmc:~#
root@amd-obmc:~#
root@amd-obmc:~# systemctl start systemd-timesyncd.service
root@amd-obmc:~#
```

## 13.4 Precautions

- Reading the RTC time may fail after AC on/off.
- The main reason is due to these two features of the ASPEED RTC
  - No battery backup supported
  - There is no default value in the RTC register.
- Due to the two aforementioned characteristics of ASPEED RTC, the RTC Register may contain unexpected initial values during AC on/off.
  - When these initial values are unreasonable for hwclock, it can cause hwclock to read these unexpected values and report an error.
  - The user only needs to write the system time to the RTC to resolve this issue.

```
root@amd-obmc:/tmp# hwclock
hwclock: RTC_RD_TIME: Invalid argument
root@amd-obmc:/tmp#
root@amd-obmc:/tmp# date
Thu Jan  1 00:31:05 UTC 1970
root@amd-obmc:/tmp#
root@amd-obmc:/tmp#
root@amd-obmc:/tmp#
root@amd-obmc:/tmp# hwclock -w
root@amd-obmc:/tmp#
root@amd-obmc:/tmp#
root@amd-obmc:/tmp# hwclock
Thu Jan  1 00:31:11 1970  0.000000 seconds
root@amd-obmc:/tmp#
```

# 14 adc_test User Guide

## 14.1 Introduction

adc_test is a tool used for reading raw data from ADC driver and estimating voltage.

Currently supported platforms are ast2600 EVB, ast2700 EVB, AMD Genoa, AMD Turin and Intel EGS.

## 14.2 Command Usage

```
root@evb-ast2600-insyde:~# adc_test

Copyfight (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

Usage: adc_test [ -s <channel> | -a ]
    -a:     show all ADC channels
    -s:     show a single ADC channel
    <channel>: ADC channel number
```

## 14.3 Example

Reading data from all of the channels by adc_test.

```
root@evb-ast2600-insyde:~# adc_test -a

Copyfight (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

Showing the reading of all ADC channels

/sys/bus/iio/devices/iio:device0
ADC_MEASURE IOCTL: channel: 0, channel name: N/A, raw data: 765, reading: 1.867676V
ADC_MEASURE IOCTL: channel: 1, channel name: N/A, raw data: 687, reading: 1.677246V
ADC_MEASURE IOCTL: channel: 2, channel name: N/A, raw data: 631, reading: 1.540527V
ADC_MEASURE IOCTL: channel: 3, channel name: N/A, raw data: 592, reading: 1.445312V
ADC_MEASURE IOCTL: channel: 4, channel name: N/A, raw data: 566, reading: 1.381836V
ADC_MEASURE IOCTL: channel: 5, channel name: N/A, raw data: 545, reading: 1.330566V
ADC_MEASURE IOCTL: channel: 6, channel name: N/A, raw data: 531, reading: 1.296387V
ADC_MEASURE IOCTL: channel: 7, channel name: N/A, raw data: 1098, reading: 2.680664V
/sys/bus/iio/devices/iio:device1
ADC_MEASURE IOCTL: channel: 8, channel name: N/A, raw data: 1021, reading: 1.196484V
ADC_MEASURE IOCTL: channel: 9, channel name: N/A, raw data: 1021, reading: 1.196484V
ADC_MEASURE IOCTL: channel: 10, channel name: N/A, raw data: 973, reading: 1.140234V
ADC_MEASURE IOCTL: channel: 11, channel name: N/A, raw data: 843, reading: 0.987891V
ADC_MEASURE IOCTL: channel: 12, channel name: N/A, raw data: 742, reading: 0.869531V
ADC_MEASURE IOCTL: channel: 13, channel name: N/A, raw data: 671, reading: 0.786328V
ADC_MEASURE IOCTL: channel: 14, channel name: N/A, raw data: 625, reading: 0.732422V
ADC_MEASURE IOCTL: channel: 15, channel name: N/A, raw data: 0, reading: 0.000000V
```

Reading data from a specific channel by adc_test.

```
root@evb-ast2600-insyde:~# adc_test -s 0
```

```
Copyfight (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

/sys/bus/iio/devices/iio:device0 ADC_MEASURE IOCTL: channel: 0, channel name: N/A, raw
data: 765, reading: 1.867676V
```